06 / 20 — Phase 6 Assessment: Data Fetching and State Ecosystems

> **Scope:** lessons 01 through 19 of Phase 6. **Version baseline:** React 19.x, TypeScript 5.x, TanStack Query v5, Redux Toolkit 2.x, Zustand 5.x, XState v5. Library-specific answers were checked on 2026-10-22 against TanStack Query 5.104, Redux Toolkit 2.13, Zustand 5.0.16 and XState 5.33 (timeline, retry, render-count and API-name answers were executed; see the changelog). Where an answer depends on a library detail, the key says so and awards credit for the *concept* even if a name or default differs by version.

This assessment does not follow the 15-section lesson template. It checks whether you can **explain**, **implement**, **debug**, and **reason** about data fetching and state, with no single "right tool" assumed.

---

1. How to take this assessment

Item	Rule
Time	4 hours total: Part 1–2 (45 min, closed notes), Parts 3–5 (3 h 15 min, official documentation allowed; no AI assistance, no copying from lessons)
Environment	Parts 3–4 are best done in a scratch Vite + React + TypeScript project with Vitest (and a mock server such as MSW, Phase 8 / 06); code should compile and your tests should run
Answers	Write answers *before* reading the key (§8). Partial credit applies; reasoning counts more than recall
Honesty	If you looked something up, say so, and do not claim the points for recall-based items

Score structure

Category	Parts	Max points
Explanation	Part 1 (15 conceptual questions × 2) + Part 2 (5 code predictions × 4)	50
Debugging	Part 3 (5 challenges × 10)	50
Implementation	Part 4 (5 tasks × 10)	50
Reasoning	Part 5 (5 architecture questions × 10)	50
Total		**200**

---

2. Part 1 — Conceptual questions (15 × 2 points)

Answer in 2–5 sentences each. Award yourself 2 points for a correct explanation including the *reason*, 1 point for a correct but shallow answer, 0 otherwise.

Q1 (Lessons 01, 03). `await fetch('/api/orders/7')` returns a response with status 500. What happens to the promise, and why does that make a "boolean `loading` / `error`" model inadequate? Name a better way to model request status.

Q2 (Lesson 02). A search box fires a request per keystroke. Explain the failure mode when responses arrive out of order, and contrast the *abort* strategy with the *ignore-stale-result* strategy. What does aborting a `fetch` guarantee about the server?

Q3 (Lesson 04). Distinguish *shape normalization* (parsing the wire format into a domain model) from *relational normalization* (id-keyed entity tables). Why is `res.json() as Order[]` not validation?

Q4 (Lesson 05). Explain "prediction versus truth" in optimistic updates. Give two properties that make an operation a *bad* candidate for optimistic UI.

Q5 (Lesson 06). "Fresh" does not mean "correct". Explain, and describe two ways a cache can show inconsistent data after a mutation even though no request failed.

Q6 (Lessons 07, 08). Name three properties of *server state* that distinguish it from UI state. Then define `staleTime` and `gcTime` and say which one removes an entry from memory.

Q7 (Lesson 09). Redux reducers must be pure, yet Redux Toolkit lets you write `state.items.push(x)` inside `createSlice`. Explain why this is legitimate, and what Redux's "event-sourced" model buys you.

Q8 (Lesson 10). Why does a Zustand selector that returns a new object or array on every call cause trouble, and how do you fix it? (State the fix(es) and note any version sensitivity.)

Q9 (Lesson 11). When a Context provider's value changes identity, which components re-render, and why can't consumers "select a slice"? Name one thing Context is genuinely better at than an external store.

Q10 (Lesson 12). What problem do finite state machines solve that three `useState` booleans (`isLoading`, `isError`, `isSuccess`) do not? What is the difference between *finite* state and *extended* state (context)?

Q11 (Lesson 13). What does `use(promise)` require of the promise's identity across re-renders, and what happens if you create the promise inside the component on every render?

Q12 (Lesson 14). Why must a server-side renderer create a new `QueryClient` (or equivalent cache) per request? What do *dehydrate* and *hydrate* accomplish?

Q13 (Lesson 15). Why is a timeout different from a failure, and how do idempotency keys make retrying a `POST` safe? Why does backoff need jitter?

Q14 (Lessons 16, 17). (a) Why is `navigator.onLine === true` not evidence that your API is reachable? (b) After a WebSocket reconnects, why must the client resync state, and what is the most robust way?

Q15 (Lessons 18, 19). (a) Give four criteria for deciding that a piece of state belongs in the URL. (b) What does "no single winner" mean in a technology selection matrix, and what should be decided *before* choosing a library?

---

3. Part 2 — Code prediction (5 × 4 points)

Work each out *before* running code. Marks are listed per item; show your reasoning.

P1 — Strict Mode and an ignore flag (4 pts)

```
function User({ id }: { id: string }) {
  const [user, setUser] = useState<UserDto | null>(null);

  useEffect(() => {
    let ignore = false;
    fetch(`/api/users/${id}`)
      .then((r) => r.json() as Promise<UserDto>)
      .then((u) => { if (!ignore) setUser(u); });
    return () => { ignore = true; };
  }, [id]);

  return <p>{user?.name ?? '…'}</p>;
}
```

In development with `<StrictMode>`, `<User id="7" />` mounts once. Assume both requests succeed. (a) How many requests are sent? (b) How many times is `setUser` called? (c) Which response does the UI display, and why? (d) How would the answers change in a production build? (1 pt each)

P2 — Query cache timeline (4 pts)

A component mounts `useQuery({ queryKey: ['t'], queryFn, staleTime: 60_000, gcTime: 10_000 })` at t = 0 s (the request completes at t ≈ 0.1 s), unmounts at t = 5 s, and remounts at t = 20 s. (a) Is a request sent at t = 20 s, and what does the component's `status` start as? (b) Repeat with `gcTime: 30_000`. (c) Repeat (b) but with the remount at t = 90 s. (2 pts for (a), 1 pt each for (b), (c))

P3 — Retry timeline (4 pts)

A `queryFn` always fails with a network error after 100 ms. Options: `retry: 2`, default `retryDelay` (exponential: 1 s, 2 s, ...). (a) How many total attempts occur? (b) At what time does `isError` become `true`? (c) What are `status` and `fetchStatus` at t = 2 s? (d) What changes if the thrown error is an `HttpError` with status 404 and `retry` is `(n, e) => !(e instanceof HttpError && e.status < 500) && n < 2`? (1 pt each) (The default delay `min(1000 * 2^failureCount, 30000)` ms and the timeline in the key were verified against TanStack Query 5.104.)

P4 — Who re-renders? (4 pts)

```tsx
// Variant 1: Context
const Ctx = createContext<{ a: number; b: number } | null>(null);
function Provider({ children }: { children: ReactNode }) {
  const [a, setA] = useState(0);
  const [b, setB] = useState(0);
  return <Ctx value={{ a, b }}>{children}</Ctx>; // and setA/setB exposed elsewhere
}
function A() { const { a } = use(Ctx)!; return <p>{a}</p>; }
function B() { const { b } = use(Ctx)!; return <p>{b}</p>; }

// Variant 2: Zustand
const useStore = create<{ a: number; b: number }>(() => ({ a: 0, b: 0 }));
function A2() { const a = useStore((s) => s.a); return <p>{a}</p>; }
function B2() { const b = useStore((s) => s.b); return <p>{b}</p>; }
```

`children` elements `<A/><B/>` (and `<A2/><B2/>`) are created by a parent that does **not** re-render. The value of `b` changes. (a) Which of `A`, `B` render in Variant 1? (b) Which of `A2`, `B2` render in Variant 2? (c) Why is the provider's `value={{ a, b }}` additionally risky when the provider's *parent* re-renders for unrelated reasons? (d) What does the store do to decide not to render `A2`? (1 pt each)

P5 — URL parsing (4 pts)

```tsx
const sp = new URLSearchParams('?tag=b&tag=a&page=abc&q=');
```

(a) `sp.get('tag')` (b) `sp.getAll('tag')` (c) `Number(sp.get('page'))` and `Number(sp.get('missing'))` (d) `sp.get('q')` vs `sp.get('missing')`. (1 pt each; (c) needs both values)

---

4. Part 3 — Debugging challenges (5 × 10 points)

For each: (1) state the **root cause** (4 pts), (2) give the **fix** with code (3 pts), (3) state how you would **validate** the fix and prevent recurrence (3 pts).

D1 — Stale title in the list (Lessons 06, 08)

```tsx
const todoKeys = { list: (f: Filters) => ['todos', 'list', f] as const, detail: (id: string)
=> ['todos', 'detail', id] as const };

const rename = useMutation({
  mutationFn: (v: { id: string; title: string }) => patchTodo(v.id, { title: v.title }),
  onSuccess: (todo) => queryClient.setQueryData(todoKeys.detail(todo.id), todo),
});
```

After renaming a todo the detail page is correct, but the list page (three filter variants visited earlier) still shows the old title until a manual reload.

D2 — "Not found" takes seven seconds; checkout duplicates orders (Lessons 15, 05)

```
// api.ts
export async function getOrder(id: string) {
  const res = await fetch(`/api/orders/${id}`);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
// OrderPage: useQuery({ queryKey: ['order', id], queryFn: () => getOrder(id) })   // library defaults
// Checkout:  useMutation({ mutationFn: createOrder, retry: 5 })                    // "fix for flaky mobile networks"
```

Users on `/orders/does-not-exist` stare at a skeleton for ~7 s before seeing an error. Support reports some customers have two identical orders.

D3 — The component that never finishes loading (Lesson 13)

```
'use client';
export function Profile({ id }: { id: string }) {
  const user = use(fetch(`/api/users/${id}`).then((r) => r.json() as Promise<UserDto>));
  return <h1>{user.name}</h1>;
}
// parent: <Suspense fallback={<Spinner />}><Profile id="7" /></Suspense>
```

The spinner shows forever; the Network panel shows a rapid stream of identical requests.

D4 — Another user's data flashes in the server-rendered HTML (Lessons 06, 14)

```
// server.tsx (Node, handles every request)
const queryClient = new QueryClient();               // module scope

export async function render(req: Request) {
  const session = await getSession(req);
  await queryClient.prefetchQuery({ queryKey: ['me'], queryFn: () => fetchMe(session.token) });
  const state = dehydrate(queryClient);
  return renderToString(<App state={state} />);
}
```

Intermittently, user A's name appears in user B's initial HTML.

D5 — Messages duplicated and the list resets (Lessons 17, 02)

```
const socket = new WebSocket('wss://example.com/chat');

function Chat() {
  const [messages, setMessages] = useState<Msg[]>([]);
  useEffect(() => {
    socket.addEventListener('message', (e) => {
      const m = JSON.parse(e.data) as Msg;
      setMessages([...messages, m]);
    });
  }, []);
  return <ul>{messages.map((m) => <li key={m.id}>{m.text}</li>)}</ul>;
}
```

After navigating away from and back to the chat page a few times, each message appears to be handled several times, and the list sometimes shows only the latest message. After a laptop sleep the list stops updating forever.

---

5. Part 4 — Implementation tasks (5 × 10 points)

Write compilable TypeScript. Include at least minimal tests where indicated. Grade by the checklists in §9.

I1 — Typed fetch wrapper (Lessons 01, 02, 15). Implement `getJson<T>(url, { signal?, timeoutMs? })` that (a) throws an `HttpError` (with `status`) for non-2xx responses, (b) rejects with the original abort/timeout error for cancellations (not wrapped), (c) rejects with a `TypeError` if a 2xx response is not JSON (e.g. captive-portal HTML), and (d) returns parsed JSON. Include four tests covering 404, abort, timeout, and HTML-200.

I2 — Optimistic toggle with rollback (Lessons 05, 06, 08). Using TanStack Query v5, implement `useToggleTodo()` that flips `done` in the `['todos','list']` cache optimistically, cancels in-flight list queries first, rolls back on error, and reconciles with the server afterward. State one limitation of snapshot-and-restore and how you would address it.

I3 — Cart store (Lessons 09–11). Implement a Zustand store with `items` (`id`, `priceCents`, `qty`), actions `add`, `remove`, `setQty`, a **derived** `selectTotalCents` selector (no stored `total`), and persistence of `items` only (versioned). Explain why storing prices from the server in persisted client state is risky and what you would store instead.

I4 — URL search-params schema (Lesson 18). Implement `parseListSearch(sp: URLSearchParams)` and `serializeListSearch(s)` for `{ q: string; sort: 'new'|'price-asc'|'price-desc'; tags: string[]; page: number }` with defaults, clamping (`page` 1–1000), de-duplicated sorted tags, defaults omitted on serialize, and never throwing. Include a round-trip test.

I5 — Submit-with-retry as an explicit state machine (Lessons 12, 15). Implement a reducer-based machine with states `idle | submitting(attempt) | waiting(attempt) | failed | succeeded`, events

`SUBMIT | OK | FAIL(transient?) | RETRY_TIMER | RESET`, and a React hook that performs the request with an `AbortSignal`, schedules the retry timer with jittered backoff, cancels on unmount, and exposes `{ state, submit, reset }`. Max 3 attempts; permanent errors fail immediately.

---

6. Part 5 — Architecture and reasoning (5 × 10 points)

Write a structured answer (typically 250–500 words) for each. Grading looks for **criteria**, **trade-offs**, **rejected alternatives**, and **conditions that would change your mind** — not a particular library.

A1 — Stack selection (Lessons 08–14, 19). A marketplace web app: server-rendered public listing and product pages (SEO), authenticated client-heavy seller dashboard (inventory tables, filters, bulk edits), cart and checkout, six engineers, REST backend, existing Redux code in the dashboard from 2019 storing API responses. Recommend a data/state architecture. Include the state inventory by owner, chosen tools per cluster, one rejected option with reasons, the seams between clusters, and two revisit triggers.

A2 — Failure policy (Lessons 15, 05). Define the retry/idempotency/UX policy for three endpoints: `GET /products`, `POST /payments`, `DELETE /cart/items/:id`. Specify retry classes, backoff, budgets, idempotency approach, user-visible behavior after exhaustion, and what you do when the outcome of the payment is unknown.

A3 — Offline level (Lesson 16). A field-inspection app (tablets, intermittent connectivity, photos, multi-hour offline, one inspector per site, supervisors edit schedules online). Choose an offline level per feature, describe the persistence and sync design, conflict policy, security considerations for shared devices, and what you would deliberately *not* build offline.

A4 — Transport decision (Lesson 17). An operations dashboard shows order status for 300 concurrent staff; business wants "real-time". Decide among polling, SSE, WebSocket (and managed services), define the event semantics (hint vs delta), resync and failure behavior, and say how you would test the "real-time requirement" claim with the stakeholders.

A5 — Migration plan (Lessons 06–08, 11, 19). Legacy app: components fetch in effects, responses are copied into a Redux store and into component state, multiple screens show the same entities and drift. Propose a staged migration to explicit ownership: sequencing, adapter boundaries, how you avoid a flag-day rewrite, how you measure progress, and what risks you would manage.

---

7. Optional self-check: where each lesson is covered

Lessons	Primary coverage
01	Q1, P1, I1
02	Q2, P1, D5, I1
03	Q1, I1 (typed failures)
04	Q3
05	Q4, D2, I2, A2

Lessons	Primary coverage
06	Q5, D1, D4, I2, A5
07	Q6, A1, A5
08	Q6, P2, P3, D1, I2
09	Q7, A1
10	Q8, P4, I3
11	Q9, P4, A5
12	Q10, I5
13	Q11, D3
14	Q12, D4, A1
15	Q13, P3, D2, I1, I5, A2
16	Q14, A3
17	Q14, D5, A4
18	Q15, P5, I4
19	Q15, A1, A5

8. Answer key

> Attempt the assessment first. Grading notes give the **expected content**; award credit for equivalent correct reasoning.

8.1 Part 1 — Conceptual answers

Q1. `fetch` resolves on any completed HTTP exchange, including 500; it rejects only on network failure or abort. The `Response` has `ok === false`, so wrapper code must check and throw a typed error. Two booleans cannot express the legal combinations (idle, loading, success, error, refreshing with stale data, error with stale data) and allow impossible combinations such as `loading && error`. Better: a discriminated union (`idle | loading | success | error`) or a library's two-axis model (`status` for "do we have data", `fetchStatus` for "is a request running"). *Wrong answers:* "fetch throws on 500" (axios habit), "add a third boolean".

Q2. Responses are not ordered by request order. A slow response for "ab" can arrive after the response for "abc", overwriting newer UI state with older results. *Abort:* an `AbortController` signal cancels the in-flight request, freeing resources and preventing the old response from ever resolving; *ignore flag:* the request completes but its result is discarded (`if (!ignore) setState`). The two are often combined. Aborting guarantees only that the *client* stops waiting; the server may already have processed (or still process) the request, so side effects are not undone.

Q3. Shape normalization turns an untrusted, inconsistent wire format (strings for dates/numbers, nulls, snake_case, optional fields, open enums) into a trustworthy domain type, validating at the boundary.

Relational normalization stores entities once by id in tables, so the same entity referenced from many lists has one copy. `as Order[]` is a compile-time assertion with no runtime effect: if the server sends something else, TypeScript is silent and the error appears later, far from the cause.

Q4. An optimistic update shows the *predicted* outcome immediately and reconciles with the server's *truth* later (confirm, replace with the authoritative response, or roll back and explain). Bad candidates: outcomes the client cannot predict (server-assigned values, validation by the server, pricing), irreversible or high-stakes effects (money movement, deletion without undo), operations with a high failure rate, and operations whose outcome depends on concurrent state others change. Credit any two with reasons.

Q5. Freshness is a policy about time; correctness is whether the data matches truth. Data can be "fresh" (within `staleTime`) and wrong if something changed on the server; it can be stale and right. After a mutation succeeds, caches can disagree if (a) the mutation updated one key (detail) but other keys (list pages with filters) hold copies of the same entity, (b) related aggregates/derived queries (counts, search results) are not invalidated, (c) an in-flight older fetch resolves after the mutation and overwrites the update, or (d) a copy was placed in component state/another store. Any two.

Q6. Server state: owned remotely, fetched asynchronously, shared, potentially stale the moment it arrives, changed by others without notice, requiring cache identity/dedupe. `staleTime`: how long fetched data is considered fresh (no refetch triggered on mount/focus); default 0 in TanStack Query. `gcTime` (formerly `cacheTime`): how long an *unobserved* entry stays in memory before removal; it removes entries (default 5 minutes). Common error: confusing the two.

Q7. Reducers are pure functions `(state, action) → newState`. RTK's `createSlice` uses Immer: the "mutating" code operates on a *draft proxy*; Immer records changes and produces a new immutable state with structural sharing, so the reducer remains externally pure. Event-sourced model: all changes are described as serializable actions processed centrally, enabling logging, time travel, undo, replay, middleware, and predictable debugging. Cost: indirection and ceremony. (Note: don't mutate and *also* return a new value from the same reducer.)

Q8. With a store built on `useSyncExternalStore` semantics, each render compares successive selector results (by `Object.is`, unless an equality function is provided). A selector returning a fresh object/array each call always looks "changed", causing extra re-renders or, in v5, potentially an infinite render loop/warning. Fixes: select primitives or stable references; select multiple fields via separate selectors; use `useShallow` (v5) or a custom equality; derive outside with `useMemo`. Behavior changed between Zustand v4 and v5 (verified: in 5.0.16 on React 19.3 such a selector threw "Maximum update depth exceeded").

Q9. When the provider's `value` changes identity (`Object.is`), *all* components consuming that context re-render, even if they use only part of the value; Context provides propagation, not subscription, and has no selector mechanism. Stores allow selector-based subscriptions so each component re-renders only when its slice changes. Context is better at *scoping/dependency injection*: different subtrees can have different provider values (theme per subtree, per-request services, per-test fixtures), and values are naturally scoped to the React tree.

Q10. Booleans permit impossible combinations and scatter transitions across handlers; a state machine makes states explicit, transitions explicit and guarded, and impossible states unrepresentable. Finite state is the qualitative mode (`idle`, `submitting`, `failed`); extended state (context) is the quantitative data (attempt count, error message, draft) that travels with a state. Credit also for: visualizability, testability, and that a `useReducer` with a discriminated union gives much of the benefit without a library.

Q11. The promise passed to `use` must have a **stable identity across re-renders** (created outside the render path, or cached by key), because React uses the promise object to track its resolution. If you create a new promise each render, every render suspends on a fresh pending promise; when it resolves React retries rendering, which creates another new promise, giving an infinite suspend loop (and repeated requests). Fix by creating the promise in a parent/server component, a cache keyed by id, or using a Suspense-aware library hook.

Q12. A module-scope cache on the server is shared by all concurrent requests and persists between them; user-specific data leaks across users (and across time). A per-request client isolates the data. *Dehydrate* serializes the cache state (queries, data, statuses) into JSON for the HTML; *hydrate* on the client restores it into a new client so the first client render matches the server HTML and avoids refetching. Both sides must agree on query keys and serialization.

Q13. A timeout means the *client* gave up; the server may have completed the operation, so the outcome is unknown. An idempotency key (a unique token per logical operation, reused across retries) lets the server recognize repeats and return the original result instead of executing again. Backoff reduces load as failure persists; **jitter** randomizes retry times so clients that failed simultaneously do not retry in synchronized waves (thundering herd).

Q14. (a) `onLine === true` means the browser is attached to some network; it cannot tell whether your API, DNS, proxy or captive portal lets requests through. `false` is reliable; `true` is a weak hint. (b) A connection can drop silently and events sent while disconnected are lost; the client cannot know what it missed. The most robust resync is to refetch the authoritative state via HTTP (or replay from a cursor/sequence number if the server supports it), treating events as hints/patches only while connected.

Q15. (a) Any four: shareable/bookmarkable; Back/Forward should restore it; refresh should preserve it; small and serializable; non-sensitive; low update frequency; useful for SSR; independent per tab. (b) The matrix maps responsibilities to tools; no tool covers all responsibilities, and fit depends on constraints (framework, team, SSR/offline needs, reversibility); the matrix informs arguments rather than computing a winner. First decide: the **ownership classification** of the state and which responsibilities are *actually* needed.

8.2 Part 2 — Predictions

P1. (a) **Two** requests in development: mount → effect (request 1) → simulated cleanup (`ignore1 = true`) → effect again (request 2). (b) **One** call: request 1's `.then` sees `ignore1 === true` and skips; request 2's sets state. (c) The user from request 2 (the second effect run), regardless of response order, because only its closure has `ignore === false`. (d) Production: one request, one `setUser` (no simulated remount). *Why:* Strict Mode's re-run of effects tests cleanup; the ignore flag makes the stale first run harmless. (The first request is still *sent*; use abort to avoid wasted work.)

P2. (a) Entry's `gcTime` of 10 s starts at unmount (t = 5) and expires at t = 15; at t = 20 the entry has been **garbage collected**, so a request is sent and `status` is `'pending'` with no data. (b) `gcTime: 30_000` : entry survives until t = 35; at t = 20 the entry exists and data (fetched at ~0.1 s) is **fresh** (`staleTime` 60 s → fresh until ~60 s), so **no request**, `status: 'success'` immediately. (c) Remount at t = 90: entry was removed at t = 35 (gc) → request, `'pending'` . (Even if it hadn't been removed, it would be stale.) *Wrong thought:* "gcTime controls freshness".

P3. (a) 3 attempts (the original + 2 retries). (b) Attempt 0 fails at 0.1 s; delay 1 s; attempt 1 starts at 1.1 s, fails at 1.2 s; delay 2 s; attempt 2 starts at 3.2 s, fails at 3.3 s; no retries left → `isError` at ≈ **3.3 s**. (c) At t = 2 s the query is waiting between attempt 1 and 2: `status: 'pending'` (no data, no final error), `fetchStatus: 'fetching'` (retry delays count as fetching in the library's model; verified in 5.104), `failureCount` 2 (verified). (d) With the custom `retry` function and a 404, the first failure returns `false` → no retry: **one attempt**, `isError` at ≈ 0.1 s.

P4. (a) **Both** `A` and `B` render: the provider value is a new object each time and both consume the context. (b) Only `B2`; `A2`'s selected slice `a` is unchanged. (c) If the provider's parent re-renders for unrelated reasons, `{ a, b }` is a new object even when `a` and `b` haven't changed, so *every* consumer re-renders unnecessarily; memoize with `useMemo` or split contexts. (d) The store compares the previous and next selector results (`Object.is` by default) for each subscribed component and only schedules a re-render when they differ.

P5. (a) `'b'` (first value). (b) `['b', 'a']`. (c) `Number('abc')` → `NaN`; `Number(null)` → `0` (a classic trap: a missing param becomes 0). (d) `sp.get('q')` → `''` (present but empty); `sp.get('missing')` → `null` (absent).

8.3 Part 3 — Debugging answers

Scoring per challenge: root cause 4, fix 3, validation/prevention 3.

D1 — Stale title in list. *Root cause:* the mutation updates only the detail key; list queries hold their own copies under keys `['todos','list', filters]` for three filters; nothing updates or invalidates them. The cache is consistent *per key*, not across keys. *Fix:* invalidate the shared prefix so active lists refetch and inactive ones refetch on next use, and keep the targeted write for the detail:

```
onSuccess: (todo) => {
  queryClient.setQueryData(todoKeys.detail(todo.id), todo);
  void queryClient.invalidateQueries({ queryKey: ['todos', 'list'] });
},
```

Alternative: `setQueriesData({ queryKey: ['todos','list'] }, old => old?.map(t => t.id === todo.id ? todo : t))` to update all cached lists immediately (careful: filtered lists might need the item added/removed, so combine with invalidation). Better to export `invalidateTodos(queryClient)` from the feature module (08 §9). *Validation/prevention:* a test that renames and then asserts both list and detail without reload (Phase 8); key factories as a single source of truth; a convention that every mutation lists the keys it affects.

D2 — Seven-second "not found"; duplicate orders. *Root cause:* (1) `getOrder` throws a generic `Error` for 404; the default retry treats it as transient and retries three times with 1+2+4 s delays (≈ 7 s) before `isError`. Retry policy ignores HTTP semantics. (2) `retry: 5` on a **non-idempotent** `POST`: when a response is lost, the retry creates a second order. Retrying without an idempotency key is unsafe because a failure/timeout means "unknown outcome". *Fix:* typed errors and a classification-driven policy:

```
class HttpError extends Error { constructor(readonly status: number) { super(`HTTP
${status}`); } }
const retryRead = (n: number, e: unknown) =>
  !(e instanceof HttpError && e.status >= 400 && e.status < 500 && e.status !== 408 &&
e.status !== 429) && n < 3;
useQuery({ queryKey: ['order', id], queryFn: () => getOrder(id), retry: retryRead });
// Checkout: no automatic retry unless the server supports Idempotency-Key; then one key per
intent:
const keyRef = useRef(crypto.randomUUID());
useMutation({ mutationFn: (cart: Cart) => createOrder(cart, keyRef.current), retry: false });
```

Offer a manual retry button that reuses the same key; reconcile unknown outcomes by reading orders before re-submitting. *Validation/prevention:* tests with 404, 503-then-200 and "dropped response after creation" (mock server executes then drops); server-side dedupe; delete duplicates; monitor requests-per-action.

D3 — Never finishes loading. *Root cause:* the promise is created *during render*. Each render creates a new pending promise; React suspends, and when the previous promise resolves it re-renders, creating another new promise, so it never reaches a render that doesn't suspend (loop with repeated requests). A component that suspends on mount also loses any state/refs created before suspending. *Fix:* make the promise identity stable: create it in a parent (or Server Component) and pass it as a prop, or read through a cache keyed by `id`, or use a Suspense-aware library:

```
// Suspense-aware hook from a data library (TanStack Query v5; `useSuspenseQuery`)
const { data: user } = useSuspenseQuery({ queryKey: ['user', id], queryFn: ({ signal }) =>
getJson<UserDto>(`/api/users/${id}`, { signal }) });
```

Validation/prevention: Network panel shows one request; a lint/code-review rule: "never call `use` on a promise created in the same component render"; React DevTools Suspense inspection; test with mocked promise resolving once.

D4 — Cross-user leak in SSR. *Root cause:* module-scope `QueryClient` shared across all requests. `prefetchQuery` for key `['me']` caches the first user's data; with `staleTime`, a later request may reuse it, or concurrent requests race on the same key and dehydrate each other's data. Singleton state on the server is shared state. *Fix:* create the client inside the request handler (per request) and dispose of it afterwards; include no user data in shared caches:

```
export async function render(req: Request) {
  const queryClient = new QueryClient({ defaultOptions: { queries: { retry: false } } });
  const session = await getSession(req);
  await queryClient.prefetchQuery({ queryKey: ['me'], queryFn: () => fetchMe(session.token)
});
  return renderToString(<App state={dehydrate(queryClient)} />);
}
```

On the client, create the `QueryClient` in `useState(() => new QueryClient())` (not module scope in SSR frameworks), and clear it on logout. Also check HTTP/CDN caching: responses with personalized HTML need `Cache-Control: private` / `no-store`. *Validation/prevention:* a test issuing two concurrent requests with different sessions and asserting HTML isolation; code review rule against module-scope mutable singletons on the server; security review.

D5 — Duplicated handlers, resetting list, dead socket. *Root cause:* (1) a module-level socket with a listener added on every mount and **never removed** → after N mounts, N handlers run per message (duplicates); (2) empty deps + `setMessages([...messages, m])` captures the **initial empty** `messages`, so each handler resets the list to `[m]`; (3) no reconnect, heartbeat, or resync: after a laptop sleep the connection dies silently and there is no recovery; plus missed messages are never fetched. *Fix:* functional update, cleanup, ownership outside the component, reconnect + resync:

```
useEffect(() => {
  const onMessage = (e: MessageEvent<string>) => {
    const m = JSON.parse(e.data) as Msg;
    setMessages((prev) => (prev.some((x) => x.id === m.id) ? prev : [...prev, m])); //
functional + dedupe by id
  };
  const unsubscribe = client.subscribe('chat:room-1', (ev) => onMessage(ev as unknown as
MessageEvent<string>));
  return unsubscribe;
}, [client]);
```

(Preferably the client provides typed events rather than raw `MessageEvent`; the key points are functional update, cleanup, shared ref-counted subscription, heartbeat with timeout, reconnect with jittered backoff, and an HTTP resync of message history on every reconnect.) *Validation/prevention:* Strict Mode in dev (double mount reveals leaks); count handlers; simulate sleep with a proxy that stops forwarding; message-dedupe by id; test reconnect with a mock server.

8.4 Part 4 — Implementation reference answers and grading checklists

I1 — `getJson`**.**

```ts
export class HttpError extends Error {
  constructor(readonly status: number, message = `HTTP ${status}`) { super(message); this.name = 'HttpError'; }
}

export async function getJson<T>(
  url: string,
  { signal, timeoutMs }: { signal?: AbortSignal; timeoutMs?: number } = {},
): Promise<T> {
  const signals = [signal, timeoutMs ? AbortSignal.timeout(timeoutMs) : undefined].filter(
    (s): s is AbortSignal => s !== undefined,
  );
  const combined = signals.length > 0 ? AbortSignal.any(signals) : undefined; // Chrome 116, Firefox 124, Safari 17.4; Node 20.3+ (MDN)
  const res = await fetch(url, { signal: combined, headers: { Accept: 'application/json' } });
  if (!res.ok) throw new HttpError(res.status);
  const type = res.headers.get('content-type') ?? '';
  if (!type.includes('application/json')) throw new TypeError(`Expected JSON but received "${type || 'no content-type'}"`);
  return (await res.json()) as T;
}
```

Tests (Vitest + `vi.stubGlobal('fetch', …)` or MSW): 404 → `rejects.toMatchObject({ status: 404 })`; abort → `rejects.toMatchObject({ name: 'AbortError' })`; timeout → `TimeoutError` (use a never-resolving handler with a short `timeoutMs`); 200 HTML → `TypeError`. *Checklist (10):* HttpError carries status (2); abort/timeout errors not wrapped (2); content-type guard (2); signals combined correctly or documented fallback when `AbortSignal.any` is missing (2); four meaningful tests (2). *Common deductions:* checking `res.ok` after `res.json()`; wrapping all errors in `NetworkError` (losing `AbortError` identity); `as T` presented as validation without comment (add: schema validation belongs here or in the caller, 04).

I2 — Optimistic toggle.

```ts
export function useToggleTodo() {
  const qc = useQueryClient();
  return useMutation({
    mutationKey: ['todos', 'toggle'],
    mutationFn: (v: { id: string; done: boolean }) => patchTodo(v.id, { done: v.done }),
    onMutate: async (v) => {
      await qc.cancelQueries({ queryKey: ['todos', 'list'] });
      const previous = qc.getQueryData<Todo[]>(['todos', 'list']);
      qc.setQueryData<Todo[]>(['todos', 'list'], (old) => old?.map((t) => (t.id === v.id ? { ...t, done: v.done } : t)));
      return { previous };
    },
    onError: (_err, _v, ctx) => {
      if (ctx?.previous) qc.setQueryData(['todos', 'list'], ctx.previous);
    },
    onSettled: () => {
      if (qc.isMutating({ mutationKey: ['todos', 'toggle'] }) === 1) {   // only the last settled mutation reconciles (verified: inside `onSettled` the settling mutation still counts, so the last one sees 1)
        void qc.invalidateQueries({ queryKey: ['todos', 'list'] });
      }
    },
  });
}
```

Limitation: whole-snapshot rollback restores the entire list to the state before *this* mutation, discarding the optimistic effects of other mutations that started later (and any server updates that arrived in between). Mitigations: roll back only the affected item (inverse patch), derive optimistic state from pending mutation *variables* (no cache write, no rollback, 05 §5.4), or serialize mutations per entity. *Checklist (10):* cancelQueries before write (2); snapshot + optimistic write via updater (2); rollback on error (2); reconciliation via invalidate (2); correct limitation + mitigation (2).

I3 — Cart store.

```ts
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

export interface CartItem { id: string; priceCents: number; qty: number }
interface CartState {
  items: CartItem[];
  add: (item: Omit<CartItem, 'qty'>) => void;
  remove: (id: string) => void;
  setQty: (id: string, qty: number) => void;
}

export const useCart = create<CartState>()(
  persist(
    (set) => ({
      items: [],
      add: (item) =>
        set((s) =>
          s.items.some((i) => i.id === item.id)
            ? { items: s.items.map((i) => (i.id === item.id ? { ...i, qty: i.qty + 1 } : i)) }
            : { items: [...s.items, { ...item, qty: 1 }] },
        ),
      remove: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),
      setQty: (id, qty) =>
        set((s) => ({
          items: qty <= 0 ? s.items.filter((i) => i.id !== id) : s.items.map((i) => (i.id ===
id ? { ...i, qty } : i)),
        })),
    }),
    {
      name: 'cart',
      version: 1,
      storage: createJSONStorage(() => localStorage),
      partialize: (s) => ({ items: s.items }),
    },
  ),
);

export const selectTotalCents = (s: CartState) => s.items.reduce((sum, i) => sum +
i.priceCents * i.qty, 0);
export const selectCount = (s: CartState) => s.items.reduce((n, i) => n + i.qty, 0);
// usage: const total = useCart(selectTotalCents);  // primitive => stable
```

Persisted prices risk: prices change on the server (and may differ per user/region); a persisted price becomes stale or tamperable and may be shown or even submitted incorrectly. Store **ids and quantities only**; look up current prices from server state (query cache) and let the server compute totals at checkout. Also: integers in cents avoid float errors; handle `version` migrations; storage unavailable (private modes)

→ `persist` with try/catch/fallback; SSR guard. *Checklist (10):* immutable updates (2); derived total via selector, no stored total (2); `partialize` + version (2); selectors return primitives (1); stale-price explanation + alternative (2); qty ≤ 0 removal edge case (1). *Common deductions:* mutating arrays in `set`; storing `total`; persisting actions.

I4 — Search params schema.

```
const SORTS = ['new', 'price-asc', 'price-desc'] as const;
type Sort = (typeof SORTS)[number];
export interface ListSearch { q: string; sort: Sort; tags: string[]; page: number }
const DEFAULTS: ListSearch = { q: '', sort: 'new', tags: [], page: 1 };

export function parseListSearch(sp: URLSearchParams): ListSearch {
  const sortRaw = sp.get('sort');
  const pageRaw = Number.parseInt(sp.get('page') ?? '', 10);
  return {
    q: (sp.get('q') ?? '').trim().slice(0, 100),
    sort: (SORTS as readonly string[]).includes(sortRaw ?? '') ? (sortRaw as Sort) :
DEFAULTS.sort,
    tags: [...new Set(sp.getAll('tag').map((t) => t.trim()).filter(Boolean))].sort(),
    page: Number.isFinite(pageRaw) ? Math.min(1000, Math.max(1, pageRaw)) : DEFAULTS.page,
  };
}

export function serializeListSearch(s: ListSearch): URLSearchParams {
  const sp = new URLSearchParams();
  if (s.q) sp.set('q', s.q);
  if (s.sort !== DEFAULTS.sort) sp.set('sort', s.sort);
  for (const t of [...s.tags].sort()) sp.append('tag', t);
  if (s.page !== DEFAULTS.page) sp.set('page', String(s.page));
  return sp;
}
```

Round-trip test: for inputs including `?tag=b&tag=a&tag=a&page=0&sort=bogus` → parse gives `{tags: ['a','b'], page:1, sort:'new'}`; `serialize(parse(x)).toString()` equals the canonical string `tag=a&tag=b`; `parse(serialize(parse(x)))` deep-equals `parse(x)`; plus `page=abc`, `page=99999`, empty input. *Checklist (10):* never throws on malformed input (2); enum validation with default (2); clamping (1); tag normalization (2); defaults omitted + fixed key order (2); round-trip test with malformed inputs (1).

I5 — Retry machine.

```
type State =
  | { s: 'idle' }
  | { s: 'submitting'; attempt: number }
  | { s: 'waiting'; attempt: number; error: string }
  | { s: 'failed'; error: string }
  | { s: 'succeeded' };
type Event =
  | { t: 'SUBMIT' } | { t: 'OK' } | { t: 'FAIL'; error: string; transient: boolean }
  | { t: 'RETRY_TIMER' } | { t: 'RESET' };

const MAX_ATTEMPTS = 3;

export function reduce(state: State, e: Event): State {
  switch (state.s) {
    case 'idle': return e.t === 'SUBMIT' ? { s: 'submitting', attempt: 1 } : state;
    case 'submitting':
      if (e.t === 'OK') return { s: 'succeeded' };
      if (e.t === 'FAIL') {
        return e.transient && state.attempt < MAX_ATTEMPTS
          ? { s: 'waiting', attempt: state.attempt, error: e.error }
          : { s: 'failed', error: e.error };
      }
      return state;
    case 'waiting': return e.t === 'RETRY_TIMER' ? { s: 'submitting', attempt: state.attempt +
1 } : e.t === 'RESET' ? { s: 'idle' } : state;
    case 'failed': return e.t === 'SUBMIT' ? { s: 'submitting', attempt: 1 } : e.t === 'RESET'
? { s: 'idle' } : state;
    case 'succeeded': return e.t === 'RESET' ? { s: 'idle' } : state;
  }
}

export function useSubmitWithRetry(send: (signal: AbortSignal) => Promise<void>, isTransient:
(e: unknown) => boolean) {
  const [state, dispatch] = useReducer(reduce, { s: 'idle' } as State);
  const attempt = state.s === 'submitting' ? state.attempt : 0;

  useEffect(() => {
    if (state.s !== 'submitting') return;
    const ctrl = new AbortController();
    send(ctrl.signal).then(
      () => dispatch({ t: 'OK' }),
      (err: unknown) => {
        if (ctrl.signal.aborted) return;                       // unmount/reset: ignore
        dispatch({ t: 'FAIL', error: err instanceof Error ? err.message : 'Error', transient:
isTransient(err) });
      },
```

```
    );
    return () => ctrl.abort();
  }, [state.s, attempt, send, isTransient]);                    // re-run for each new attempt

  useEffect(() => {
    if (state.s !== 'waiting') return;
    const delay = Math.random() * Math.min(8000, 500 * 2 ** (state.attempt - 1)); // full
jitter
    const id = setTimeout(() => dispatch({ t: 'RETRY_TIMER' }), delay);
    return () => clearTimeout(id);
  }, [state]);

  return { state, submit: () => dispatch({ t: 'SUBMIT' }), reset: () => dispatch({ t: 'RESET'
}) };
}
```

Notes for grading: effects are keyed to *state entry* (no stale `isLoading` flags); the request effect's cleanup aborts on state exit/unmount; `send` and `isTransient` should be stable (`useCallback`) or held in refs, otherwise the effect restarts each render (a deduction if unaddressed). Strict Mode's simulated remount aborts and re-sends the first attempt in development; the abort guard avoids dispatching stale failures. *Checklist (10):* exhaustive, impossible-state-free union (2); transitions correct incl. permanent error → failed immediately (2); abort + cleanup (2); jittered backoff via timer effect (2); max attempts respected (1); stable function deps addressed (1).

8.5 Part 5 — Architecture: evaluation criteria and model answers

Scoring for each (10 pts): state/responsibility analysis (3), justified recommendation with seams (3), rejected alternatives + trade-offs (2), revisit conditions / risks (2).

A1 — Marketplace stack. *Model answer.* Inventory by owner: public catalog (server state; SEO; non-interactive) → framework server-rendering/loader layer (or server components if the framework offers them); filters/sort/pagination → URL; seller dashboard tables (server state, interactive, shared across widgets) → client server-state cache (TanStack Query, or RTK Query if the dashboard remains Redux-centered); cart (server-authoritative with optimistic UI; persisted guest cart in storage) → client cache with idempotent mutations; checkout flow → reducer/state machine; ephemeral UI → local state; small UI preferences → Context or a small store. Seams: URL → query keys; mutations → targeted invalidation; no entity copies in Redux. Legacy Redux: stop adding server entities to Redux; migrate dashboard queries to the cache library behind feature `api` modules; keep Redux only for genuinely client-owned state until it can be removed. Rejected: "RTK Query everywhere" — acceptable if the team is Redux-fluent (rejected here because the public pages are server-rendered, and expanding Redux increases coupling); "global store for all state" — duplicates cache responsibilities. Revisit triggers: framework change (Server Components/loaders ownership), GraphQL gateway adoption, offline requirement, shared mutable entities across many views requiring normalization. Credit also for per-request cache isolation on the server and idempotent checkout.

A2 — Failure policy. *Model answer.* `GET /products`: idempotent; retry transient only (network, timeout, 408, 429, 502–504; optionally 500 with budget) up to 2–3 times, full jitter (base ~0.5–1 s, cap ~8 s), honor

`Retry-After`; show skeleton → "Still trying…" after first failure → error with manual retry; keep stale data. `POST /payments`: **no automatic retry without server idempotency**; with an idempotency key generated per intent (persisted for reload) a bounded retry is safe; on unknown outcome after exhaustion/timeouts: show "We couldn't confirm your payment", reconcile via `GET` by key/order before allowing a new attempt, never create a new key for the same intent; ensure server enforces payload match for a key. `DELETE /cart/items/:id`: idempotent in effect; 1–2 automatic retries; treat 404-on-retry as success; optimistic removal with rollback. Cross-cutting: single retry owner (avoid layering), retry budgets, metrics (success-after-retry), central error reporting. Credit: distinguishing transient vs permanent, handling 401 once, UX copy.

A3 — Offline level. *Model answer.* Level 3 reads for assigned inspections/checklists (persisted cache or local DB with `maxAge`, "as of" labels); Level 4 writes for checklist answers and photos via an outbox in IndexedDB with idempotency keys, ordered replay, resumable uploads for photos (Blobs, quota management, compress/limit); conflict policy: one inspector per site so conflicts are rare — field-level merge for checklist items, optimistic concurrency (`If-Match`/version) for supervisor-edited schedule data with a visible resolution UI; supervisors' schedule edits stay online-only. Shared devices: per-user storage keyed/busted by user id, wipe on logout, encryption considerations, no refresh tokens in script-readable storage, remote wipe/expiry for abandoned devices. Do *not* build: offline editing of schedules, offline user management, multi-device merge/CRDT (unneeded), offline reporting/analytics. Include service worker for app shell, kill switch, update flow, and tests with network disabled. Credit: explicitly choosing what not to build and why; sync-on-open as baseline since Background Sync support varies.

A4 — Transport decision. *Model answer.* First clarify the requirement: acceptable staleness (seconds?), change rate, user count (300), and what actions follow. Likely sufficient: polling every 5–10 s with `refetchInterval` (paused when hidden) or SSE with invalidation hints; WebSocket only if bidirectional low-latency features are needed. Event semantics: hints (invalidate order-list/detail queries) with HTTP as source of truth; deltas only for hot fields with versions; always resync on reconnect via refetch; heartbeat and jittered reconnect; per-topic ref counting; server fan-out and graceful draining at deploy; auth via short-lived tokens/cookies and origin checks. Validate the claim with stakeholders: measure how quickly staff act on changes, A/B a 10 s polling prototype, define the SLO ("status visible within 10 s for 99%"). Trade-offs: polling cost (300 users × 6 req/min = 1,800 req/min — trivial with caching) vs operational complexity of persistent connections. Credit: not defaulting to WebSockets, handling failure/resync, testing the requirement.

A5 — Migration plan. *Model answer.* Stage 0: audit (ownership audit for 3–5 key facts; count fetch patterns; baseline defects/requests). Stage 1: introduce the server-state cache and a thin `features/*/api` layer (query-key factories, `queryOptions`, central retry/error policy) with lint rules to stop *new* effect-fetches and new server entities in Redux. Stage 2: migrate by feature slice starting with the screens that exhibit drift (strangler pattern): replace effect-fetch+copy with query hooks; delete the Redux/component-state copies once consumers are switched; keep the Redux shape temporarily through a read-adapter if many selectors depend on it. Stage 3: URL state for filters; invalidation helpers per feature; add tests per migrated slice (MSW). Stage 4: remove dead Redux slices; evaluate whether Redux remains for client state. Measure: count of `useEffect+fetch`, duplicate-source facts, requests per page view, bug rate for stale data, bundle delta. Risks: temporary dual systems (limit lifetime), key design mistakes (review early), team learning curve (templates, pairing), SSR/hydration if present, regressions (feature flags per screen). Credit: avoids flag-day rewrite, defines adapters and measurable progress, addresses coexistence period.

---

9. Scoring sheet and rubric

9.1 Scoring sheet

Category	Max	Your score	%	Pass (≥ 70%)
Explanation (Part 1: ___/30 + Part 2: ___/20)	50			
Debugging (Part 3)	50			
Implementation (Part 4)	50			
Reasoning (Part 5)	50			
Overall	**200**			**Pass (≥ 80%)**

Pass condition: overall **≥ 160 / 200 (80%)** and **each category ≥ 35 / 50 (70%)**.

9.2 Category rubrics (what each score band looks like)

Band	Explanation	Debugging	Implementation	Reasoning
90–100%	Explains *why* using first principles (event loop, identity, ownership); predicts timelines precisely; names version-sensitive items	Finds root cause fast; fix addresses cause and prevents recurrence; validation uses concrete DevTools/test evidence	Code compiles, handles edge cases (abort, errors, identity), minimal and idiomatic; tests meaningful	Frames by responsibilities and constraints; presents rejected options and reversal triggers; quantifies where possible
80–89%	Correct with minor omissions in reasoning	Correct cause, partly complete fix or validation	Works; misses one edge case or test	Solid recommendation; weaker on alternatives or triggers
70–79%	Mostly recall; some reasoning gaps	Treats symptoms; fix works but root cause partly misdiagnosed	Works for happy path; weak error/cancel handling	Picks a tool with partial justification; ignores constraints
< 70%	Misconceptions remain (e.g. fetch throws on 500; `gcTime` = freshness; Context is a store)	Guess-and-patch; no validation	Does not compile or major bugs (races, leaks, duplicated state)	Tool-first answers ("use X"); no trade-offs

Penalty notes. Deduct up to 2 points per task for: unsafe retry of non-idempotent operations, copying server data into component/client store without justification, missing cleanup of subscriptions/timers/requests, and treating URL input as trusted. Do not award credit for confident statements of facts that are wrong (e.g. invented defaults); credit "I would verify X in the docs" only with a correct reasoning path.

9.3 Mapping scores to remediation

If your weakest area is	Re-study	Redo
Explanation of requests, races, status	01, 02, 03	Q1–Q2, P1, D5
Data shaping and optimism	04, 05	Q3–Q4, I2

If your weakest area is	Re-study	Redo
Caching and invalidation	06, 07, 08	Q5–Q6, P2, D1, I2
Client state tools	09, 10, 11, 12	Q7–Q10, P4, I3, I5
Suspense and SSR data	13, 14	Q11–Q12, D3, D4
Resilience, offline, real-time	15, 16, 17	Q13–Q14, P3, D2, D5, A2–A4
URL state and decision-making	18, 19	Q15, P5, I4, A1, A5

10. Promotion criteria

You are ready to proceed to Phase 7 — Performance (which assumes this phase's data/state vocabulary, especially cache behavior, selectors, and re-render causes) when **all** hold:

1. **Score:** overall ≥ 80% and every category ≥ 70% (§9.1).
2. **No critical misconceptions** in the "penalty notes": you must not (a) retry non-idempotent operations without an idempotency strategy, (b) store server state as client state by default, (c) leave subscriptions/requests uncleaned, or (d) trust URL/network input unvalidated.
3. **Project evidence:** Project 6 (E-commerce with server-state caching, in `../projects/`) is complete and passes its review checklist, demonstrating: URL-driven query keys, targeted invalidation, an optimistic mutation with rollback, a typed error taxonomy with a retry policy, and a decision record (ADR) for the chosen stack.
4. **Explain-aloud test:** you can, without notes, walk a peer through (a) the lifecycle of a request with a race, (b) a cache invalidation after a mutation, and (c) the justification for one stack choice *and the strongest alternative* in under ten minutes.
5. **Debug drill:** given an unfamiliar bug report in this domain (stale data, duplicate requests, hydration mismatch, listener leak), you locate the root cause with browser DevTools and Query/Redux DevTools in under thirty minutes.

If you pass the score but not criteria 3–5, complete those before moving on: Phase 7 assumes you can read Network and Profiler timelines for data-heavy interfaces.

Retake policy: wait at least 48 hours, revisit the remediation table (§9.3), and re-attempt *new* variants of failed items (change the scenario while keeping the concept) rather than re-reading the key.